

Informe de Exposiciones: Herramientas de Modelado y Generación de Código

Grupo H: Enterprise Architect

Huilcapi

Teórica: Presentó la herramienta como un software de modelado profesional que ayuda a analizar y documentar software y arquitectura empresarial, disponible para uso académico. Explicó los estándares que soporta: UML, BPMN y SysML, detallando cada uno de ellos.

Práctica: Mostró el diagrama de clases del proyecto de palma africana, creó la clase GestionInsumos, explicó dónde asignarle atributos y operaciones, agregó los atributos IDInsumos y nombreInsumos, y creó las operaciones actualizarInsumo() y consultarInsumo(). Estableció una relación de asociación entre GestionInsumos e Insumos, explicando cómo nombrar la relación y asignar la cardinalidad (1 a N), y mostró el resultado final de la clase con su relación.

Vaca

Teórica: Explicó el funcionamiento de la herramienta y el ciclo de modelado integral, el repositorio central que permite el trabajo colaborativo, y los tipos de diagramas soportados (casos de uso, actividad, secuencia, clases, componentes, despliegue y estados), detallando en qué ayuda cada uno. Describió los tres pilares de la herramienta: gestión y trazabilidad (trazabilidad completa, repositorio centralizado, matriz de trazabilidad), documentación e ingeniería, y simulación y colaboración.

Práctica: Explicó la interfaz del programa, cómo crear un nuevo proyecto con la opción Create New, el nivel de seguridad de acceso, la asignación de nombre a un paquete (diagrama) y la selección del estándar a usar (en su caso, UML estructural). Mostró dónde encontrar la toolbox y explicó que con la tecla F9 se pueden visualizar los atributos, además de las relaciones del diagrama.

Tigasi

Teórica: Expuso las ventajas de la herramienta: capacidad de manejar el análisis de requerimientos en una sola plataforma y alto rendimiento para trabajar en proyectos grandes y multidisciplinarios, respaldada por la comunidad y con soporte tanto para universidades como para empresas. Mencionó limitaciones como la curva de aprendizaje alta por la cantidad de funciones, el costo de licencia y la necesidad de instalación de escritorio. Destacó tres factores clave: estándares, adaptación en la educación universitaria y uso empresarial en sectores como tecnología, finanzas, salud y gobierno. Concluyó que es una herramienta completa que permite la generación de código con trazabilidad, considerada de primer nivel para el desarrollo de proyectos empresariales a gran escala.

Práctica: Explicó cómo generar el código en C#, incluyendo el cambio de lenguaje, la selección de la carpeta de destino y la generación del código para todas las clases, mostrando cada una con

su respectivo código. Explicó el uso de ingeniería inversa (modificar el diagrama a partir de cambios en el código) mediante la función de sincronización, seleccionando la clase afectada y mostrando el resultado. Mostró también la generación de un informe de los diagramas, con selección de idioma y carpeta de almacenamiento, así como los formatos de exportación disponibles y la trazabilidad de cada clase.

Grupo A: StarUML

Díaz

Teórica: Explicó los fundamentos del desarrollo dirigido por modelos (MDD): un enfoque en el que primero se elabora el diagrama y el código se guía a partir de él. Describió el modelo MDA con sus tres niveles (CIM, independiente de la computación; PIM, donde se realizan los diagramas; y PSM, donde al modelo se le asigna una tecnología o lenguaje), así como las transformaciones M2M (modelo a modelo) y M2T (modelo a código). Presentó StarUML como un software centrado en el análisis y diseño de diagramas, con soporte de extensiones para la generación de código en múltiples lenguajes.

Práctica: Mostró un proyecto ya creado, su diagrama de clases, dónde se puede generar el código, y la extensión de C# instalada.

Escudero

Teórica: Justificó la elección de StarUML por su compatibilidad con UML 2.5, la generación automática de código (en este caso C#), la compatibilidad con .NET, la sencillez de su interfaz y el mantenimiento de la trazabilidad entre modelo y código. Presentó el subsistema de gestión de un gimnasio mediante un flujograma con las etapas de registro de clientes, gestión de planes, gestión de membresías, registro de pagos y emisión de comprobantes.

Práctica: Demostró la trazabilidad copiando y modificando un método de la clase Membresías, y guardó el cambio mediante la función roundtrip, mostrando el resultado reflejado en el código.

Mera

Teórica: Explicó el modelado UML sobre un diagrama ya creado, señalando que las clases presentan generalización y herencia (Persona), que Comprobante tiene una relación de agregación con Pago, y describiendo cardinalidades y operaciones de cada clase. Como ventajas mencionó que es multiplataforma, útil para el proceso de modelado, con opciones de exportación y amigable; como desventaja, que siendo de código abierto la versión utilizada es gratuita y las funciones avanzadas requieren licencia de pago. Explicó el proceso de generación de código y concluyó que es una herramienta versátil.

Práctica: Mostró las clases guardadas y, en Visual Studio, las agregó como elemento existente a partir del diagrama, verificando en el explorador de archivos que se almacenaran correctamente.

Mesías

Teórica: Describió limitaciones como la restricción de funciones hasta obtener licencia, la curva de aprendizaje y la incompatibilidad con otras aplicaciones. Realizó una comparativa funcional frente a otras herramientas, señalando un complemento adicional propio y comparándola con Visual Paradigm, destacando que StarUML es más ligera y fácil de aprender. Explicó la generación de código mediante una extensión oficial y el uso del archivo de proyecto propio.

Práctica: Generó el código en C# seleccionando todas las clases del diagrama y la carpeta de destino, y en Visual Studio las agregó como elemento existente, verificando en el explorador de archivos que se guardaran correctamente.

Grupo G: UMPLE Online

El grupo trabajó sobre un proyecto de camaronera.

Alvia

Teórica: Explicó que UMPLE es una herramienta para programación orientada a modelos, con sincronización bidireccional en vivo entre el código (archivo .ump) y el diagrama generado, eliminando por completo las redundancias. Mostró los diagramas soportados por la plataforma y explicó que ayuda a reducir el tiempo de desarrollo, siendo una plataforma cómoda de usar.

Práctica: Trabajó con el archivo .ump generando código mientras se creaba el diagrama en paralelo, y generó un diagrama de estados a partir de código en Java.

Arboleda

Teórica: Presentó las características de UMPLE: modelado estático, representación intuitiva de clases, herencia múltiple simulada, interfaces, máquinas de estado y trazabilidad.

Práctica: Mostró cómo se plasma un diagrama de clases mediante código, copiando y pegando el código y generando el diagrama correspondiente.

Calle

Teórica: Describió el ciclo de desarrollo de UMPLE Online: concepción y diseño rápido del modelo textual, sincronización (el diagrama se actualiza en tiempo real), simulación (prueba del comportamiento real) y exportación. Destacó la facilidad de uso, que permite a los estudiantes generar código o diagramas a partir de clases.

Práctica: Realizó una práctica equivalente a la de sus compañeros, trabajando con otro ejemplo del proyecto.

Grupo E: Eclipse Papyrus + Acceleo

Pérez

Teórica: Explicó los niveles de abstracción CIM (procesos y requisitos sin definir tecnología, modelo conceptual), PIM (punto de vista de diseño sin especificar lenguaje o dispositivo) y PSM (adaptación a un lenguaje, framework o base de datos ya definidos). Describió las transformaciones en MDD: M2M (PIM a PSM), M2T (modelo a código), forward (generación

en un solo sentido) y roundtrip. Justificó el uso conjunto de Papyrus (para crear diagramas UML) y Acceleo por ser ambas de código libre, basarse en estándares OMG, integrarse en el mismo entorno Eclipse y permitir personalización de plantillas según cada proyecto. Entre los beneficios del MDD mencionó la automatización (menos trabajo manual), la mayor productividad, la claridad del código, la trazabilidad, el mantenimiento de cambios desde el modelo y la mejora en la calidad. Concluyó que es una combinación excelente que eleva la calidad y agilidad del desarrollo de software.

Práctica: Seleccionó la carpeta de recursos donde se genera el código, solucionó un error de la plantilla, la importó y configuró, y mostró las tablas con sus atributos y métodos junto con el código Java generado a partir de la plantilla.

Gamarra

Teórica: Explicó que MDE es el enfoque general de ingeniería basado en modelos y que MDA es el estándar OMG con los niveles CIM, PIM y PSM. Presentó Eclipse Papyrus como herramienta de modelado UML que permite crear y editar diagramas para análisis y diseño, compatible con los estándares OMG, con ventajas como ser software libre, extensible e integrable. Describió Acceleo como el complemento que, dentro del mismo entorno, transforma el diagrama creado en código a partir de plantillas que definen cómo debe traducirse (MOFM2T).

Práctica: Mostró la interfaz del programa, creó un nuevo proyecto de Acceleo, eligió el estándar UML 2.5, configuró el entorno de Acceleo con Papyrus e implementó la plantilla, generando el código en Java a partir de las clases. Demostró que un cambio realizado en el diagrama (agregar un atributo) se refleja en el código actualizado.

Grupo B: BOUML

Mi grupo.

Grupo D: Modelio

El grupo trabajó sobre un proyecto de camaronera.

Rizzo

Teórica: Presentó Modelio como herramienta que soporta los estándares UML y BPMN, con verificación de consistencia mediante auditorías automáticas y arquitectura modular extensible. Relató su historia y evolución: Objecteering como precursor en 1991, que posteriormente añadió soporte para MDA y UML; en 2011 Modelio se lanzó como código abierto, e incorpora actualmente funciones de ingeniería inversa, entre otras. Entre las características principales destacó el modelado UML completo con chequeo de consistencia, el soporte multiestándar (UML2, BPMN2, ArchiMate), la organización jerárquica de proyectos, la arquitectura modular extensible y su naturaleza multiplataforma.

Práctica: Mostró la interfaz del programa, los módulos compatibles, la jerarquización de los proyectos creados con anterioridad y un diagrama de clases ya elaborado.

Crespo

Teórica: Describió la interfaz, similar a las anteriores (explorador del modelo, área de trabajo, paleta de herramientas y vista de propiedades), y los diagramas disponibles: estructurales, de comportamiento, actividad, estado, proceso de negocio, diccionario de datos y arquitectura, además del módulo ArchiMate para ampliar las opciones de diagramación. Explicó los estereotipos y elementos de modelado (dominio, identidad, código en Java).

Práctica: Mostró el apartado donde se genera el código, el desglose del código generado y las opciones de personalización, seleccionó la ruta de almacenamiento y las opciones de cada clase (atributos), y mostró los elementos para la creación de diagramas. Generó el código seleccionando la carpeta correspondiente y verificó en el explorador de archivos las clases creadas junto a sus atributos y operaciones.

Amagua

Teórica: Enumeró ventajas (núcleo open source, soporte UML2/BPMN2/ArchiMate, arquitectura modular y extensible, comunidad activa —mayormente inglesa y francesa—, más de 20 años de trayectoria y naturaleza multiplataforma) y desventajas (curva de aprendizaje pronunciada, funciones avanzadas de pago, no preservación del código escrito manualmente y documentación en inglés y francés). Justificó la selección de la herramienta por restricción presupuestaria y su capacidad de generación de código Java integrada. Explicó los beneficios del MDD (generación de código desde modelos UML, reducción del tiempo de desarrollo, disminución de errores, facilidad de documentación y mejora del mantenimiento y la reutilización) y sus limitaciones (ajustes manuales requeridos, traducción incompleta de elementos UML a código, necesidad de conocimiento de UML y riesgo de pérdida de cambios no guardados). Concluyó que es una herramienta madura, open source y adecuada para el aprendizaje.

Práctica: Mostró un diagrama de estados creado previamente, correspondiente al requisito número 5 de la planificación de la siembra, explicando el flujo del diagrama y las relaciones entre estados.

Grupo C: Visual Paradigm

Castro

Teórica: Explicó los modelos de software: MDE, donde los modelos dejan de ser solo documentación y pasan a ser la fuente principal para derivar artefactos; MDA, con sus niveles CIM, PIM y PSM; y MDD, aplicado al desarrollo. Explicó los perfiles y estereotipos UML: entity, service y repository.

Práctica: Explicó los estereotipos y la validación, mostrando los aplicados a la clase Palma y la clasificación de sus atributos con sus características.

Mora

Teórica: Describió la configuración del generador de código: lenguaje objetivo Java, codificación UTF-8 y compatibilidad con JDK 8, mapeo de asociaciones múltiples, carpeta de salida donde se almacenan los diagramas y la política de regeneración.

Práctica: Utilizó el Instant Generator, verificando que todos los elementos estuvieran seleccionados, eligió la opción array para generar el código, seleccionó Java y la codificación UTF-8, y el directorio de destino. Generó primero una vista previa para revisar las clases generadas y, al no encontrar errores, procedió a generar el código final, mostrando en el directorio las clases generadas, todas con el estereotipo entity.

Vera

Teórica: Explicó la generación y estructura del proyecto, aclarando que al generar el código no se implementa automáticamente la lógica de negocio. Detalló qué debe verificarse manualmente tras la generación: reglas de negocio, persistencia real, manejo de errores y pruebas unitarias.

Práctica: Explicó que la aplicación tiene restricciones con C#: el programa permite generar en Java libremente, y en C# solo con permisos de administrador. Mostró el código generado como clase estructurada, modificó la clase Palma agregando el atributo público fechaUltimoMantenimiento, guardó el cambio, creó un nuevo directorio y generó el código en Java, mostrando la clase modificada con el atributo agregado.

Villafuerte

Teórica: Justificó el uso del proyecto Sigma porque cumple con el requisito de pasar de diagrama a código, se guarda en formato .vpp, cuenta con Instant Generator para Java, permite estereotipos UML que reducen el riesgo de migración y ofrece mayor trazabilidad.

Práctica: Mostró la interfaz del programa y un diagrama de clases creado anteriormente del proyecto Sigma, explicó cómo crear un nuevo diagrama y listó las ocho clases ya creadas, nombrando sus relaciones (composición entre Lote y Palma, herencia entre Usuario y Trabajador, entre otras), sus propiedades y métodos, así como los elementos para crear un nuevo diagrama y la pestaña para agregar atributos y operaciones.

Nota: ningún grupo presentó los requisitos correspondientes durante la exposición.